# Ticket1: Software Test Plan (STP)

CS 4223 - Software Quality and Testing

Spring 2026

Group C - Cluster 1

Colby Frison, Joy Mosisa, Johnpaul Nguyen, Leah Tchatchoua, Trinity Tran, Harry Uwakwe

Due: April 1, 2026

**Software Test Plan (STP) - Ticket 1 ..... 2**
Scope ..... 2
1. Introduction ..... 2
1.1 Component overview ..... 2
1.2 Scope and objectives of testing ..... 3
1.3 Assumptions and constraints ..... 3
2. Quality Attributes & Risks ..... 3
2.1 Selected quality attributes ..... 3
2.2 Rationale for why these attributes are critical ..... 4
2.3 Identified quality risks ..... 4
3. Test Strategy ..... 4
3.1 Overall testing approach ..... 4
3.2 Mapping test types to quality attributes ..... 5
3.3 Black-box, white-box, and object-oriented considerations ..... 5
3.4 Test prioritization and automation ..... 6
4. Verification & Validation Approach ..... 6
4.1 How verification and validation are conducted ..... 6
4.2 Alignment with software lifecycle ..... 6
4.3 Traceability to expected behavior ..... 7
4.4 Standards alignment ..... 7
5. Test Environment ..... 7
5.1 Tools, frameworks, and configuration ..... 7
5.2 Test data considerations ..... 7
6. Test Coverage & Metrics ..... 8
6.1 Coverage criteria ..... 8
6.2 Metrics used to evaluate test effectiveness ..... 8
7. Regression & Nonfunctional Testing Strategy ..... 8
7.1 Regression testing approach ..... 8
7.2 Planned nonfunctional tests ..... 9
8. Limitations & Risks ..... 9
8.1 Known limitations of the test plan ..... 9
8.2 Potential challenges and mitigation strategies ..... 10
**Reference ..... 11**

# Software Test Plan (STP) - Ticket 1

**System Under Test:** *OpenPilot/openpilot-0.9.8/*

## Scope

In the openpilot repository, we will be creating a testing plan for the following directories:

- *selfdrive/modeld* (all files)
- *selfdrive/pandad* (assignment-listed files only):
    - *pandad_api_impl.pyx*
    - *pandad.cc*
    - *pandad.h*
    - *pandad.py*
    - *SConscript*
    - *spi.cc*
- *system/* (entire directory)

The purpose of this STP is to explain how testing should be organized and why it is justified for this scope. Existing tests in the repository are treated as evidence of the current state of the codebase, this document provides structure, priorities, risks, limitations, and acceptance logic for the testing present in the repository.

## 1. Introduction

### 1.1 Component overview

Openpilot is a process-oriented autonomous driving software stack. Instead of a single monolithic executable, it acts as cooperating services that exchange data continuously to work together. For this ticket, we will be testing within the three directories/subdirectories listed above. We broke these down into three main layers to make the testing plan more manageable.

The first layer is *selfdrive/modeld*, which runs the model pipeline and publishes outputs such as *modelV2* and *cameraOdometry*. The second is the subset of *selfdrive/pandad*, which connects host software to Panda transport/safety interaction points. The third is *system/*, which is the broad runtime platform that manages orchestration, logging, camera paths, sensors, networking, updates, and hardware integration. Due to the complexity of this third *system/* layer it appears more in the testing plan than the other two to accommodate its larger amount of dependencies and subsystems.

Because these layers are tightly coupled at runtime, the STP uses a mixed strategy of focusing on component tests, interface tests, and lifecycle tests across process boundaries in order to provide a comprehensive test plan.

### 1.2 Scope and objectives of testing

The overall objective is to provide confidence that in-scope behavior remains correct, stable, and operationally safe under realistic conditions. The primary five goals we will be focusing on are:

- Accuracy of outputs and side effects;
- Safety-aligned behavior at process and transport boundaries;
- Reliability under restart, reconnect, and degraded conditions;
- Acceptable performance/timeliness in critical paths;
- Appropriate security/privacy behavior in network-facing and upload-related services.

This artifact is planning what to do and verifying what has been done; it is not a replacement for implementation-level tests, and should be used in conjunction with the existing tests in the repository.

### 1.3 Assumptions and constraints

This plan assumes the same code can behave differently across runtime contexts, especially across different hardware configurations. Device class (desktop vs. comma 4), sensor availability, and transport mode (type of car) can change the observable behavior and process graph.

The main constraints are:

- Hardware-bound tests cannot be fully represented by PC-only CI;
- Model assets are treated as baseline artifacts, not retraining targets;
- Native and Cython builds must pass for in-scope areas;
- *pandad* commitments are limited to the specific files listed, while dependencies may still be referenced for context.

## 2. Quality Attributes & Risks

### 2.1 Selected quality attributes

The quality focus is intentionally practical and extremely safe since it is for autonomous driving runtime software. In this scope, failures rarely stay isolated: one subsystem issue can cascade through messaging and process orchestration, which can lead to difficult troubleshooting and potential safety issues. The main selected quality attributes are:

- Functional correctness;
- Safety and integrity;
- Reliability and availability;
- Performance and timeliness;
- Maintainability and verifiability;
- Security and privacy.

These attributes directly map to the observed design of *modeld*, *pandad*, and the *system/* runtime platform.

## 2.2 Rationale for why these attributes are critical

Each selected attribute is important for the safety and reliability of the system for the following reasons:

- **Functional correctness:** *modeld* and surrounding services feed downstream planning/control logic, so incorrect outputs can quickly build up.
- **Safety and integrity:** Software that directly interacts with the Panda and processes that gate the Panda are safety-relevant boundaries where subtle defects have a disproportionate impact, as pandad directly interacts with the Panda hardware, which is critical to ensuring correct behavior.
- **Reliability and availability:** The platform is process-based, so quality depends on stable startup, restart, and recovery behavior across many daemons.
- **Performance and timeliness:** Camera, model, and logging paths are time-sensitive; latency spikes or backlog can degrade operational behavior, and in a system where "where timing is critical, delays can significantly impact system behavior for the stakeholders.
- **Maintainability and verifiability:** The mixed Python/C++/Cython stack increases complexity, so ensuring the tests and the surrounding system are well written and maintained is crucial to ensure the system is easy to understand and modify. And without verification, truly saying something is correct is impossible.
- **Security and privacy:** *athena*, *webrtc*, and upload/logging paths expose network and data surfaces that must fail safely, ensuring privacy and safety in the event of malicious behavior.

## 2.3 Identified quality risks

The following risks are treated as high-priority due to the nature of the system, the potential impact of the failures, and their role in ensuring the quality of the system:

- R1: Stale or mismatched frame/output behavior in *modeld*;
- R2: Incorrect safety mode behavior around pandad interactions;
- R3: Transport boundary failures across *spi.cc* and *pandad_api_impl.pyx*;
- R4: Process orchestration failures in *system/manager*;
- R5: Log corruption, drops, or disk pressure failures in *system/loggerd*;
- R6: Session/auth misuse in *system/athena* and *system/webrtc*;
- R7: Camera instability causing downstream inference issues;
- R8: Update flow integrity failures in *system/updated*;
- R9: GPS/sensor inconsistency under hardware variability;
- R10: Thermal/power policy misbehavior in *system/hardware*.

# 3. Test Strategy

## 3.1 Overall testing approach

The strategy is organized into four test levels, each targeting a different class of defect. Every level draws from all three in-scope layers (*modeld*, *pandad*, *system/*).

- **Unit tests** verify isolated logic with no external dependencies. *test_pandad_usbprotocol.cc* tests CAN packing/unpacking across 96 parameterized configurations in memory. *test_deleter.py*

verifies log deletion order using a mocked filesystem. In *modeld*, *parse_model_outputs.py*'s Parser class and its sigmoid/softmax utilities are self-contained candidates for unit verification.

- **Integration tests** verify communication across process and serialization boundaries. *test_pandad_loopback.py* sends random CAN messages through the full pandad pipeline and asserts zero message loss over 200 iterations. *test_modeld.py* starts a fake camera server, sends synthetic frames, and verifies output messages have correct frame IDs, timestamps, and zero frame drops. *test_loggerd.py* publishes random cereal messages and confirms every one is recorded with correct sentinels.
- **System tests** verify end-to-end lifecycle behavior. *test_pandad.py* tests firmware recovery from DFU mode, bootstub mode, and corrupted firmware on real hardware. *test_manager.py* starts all managed processes and verifies each exits cleanly. *test_athenad.py* tests remote command dispatch using mocked websockets.
- **Stress and fault-injection tests** verify behavior under degraded conditions. *test_pandad_spi.py* activates built-in fault injection (SPI_ERR_PROB=0.001) that randomly corrupts SPI transfers, then asserts message integrity, valid hardware state, and publishing frequency within 10% tolerance. *test_camerad.py* asserts frame-to-frame timing stays within 1ms over 10 seconds of continuous operation.

## 3.2 Mapping test types to quality attributes

Each subsystem's tests map to multiple quality attributes rather than a single one:

| Subsystem | Correctness | Safety | Reliability | Performance | Maintainability |
|---|---|---|---|---|---|
| *modeld* | Frame ID/output verification (*test_modeId*) | Dropped-frame withholding (*test_modeld*) | (*Indirectly covered by test_manager.py, test_modeld.py*) | Model execution timing | Build via *SConscript* |
| *pandad* | Parameterized pack/unpack (*test_usbprotocol*) | Staleness check, heartbeat, safety mode enforcement | Firmware recovery, zero message loss (*test_pandad*, *test_loopback*) | Startup time benchmark (*test_pandad*) | Cython/C++ build chain |
| *system/* | Log record completeness (*test_loggerd*) | (*Indirectly covered by pandad*) | Process clean exit (*test_manager*), deletion order (*test_deleter*) | Camera frame timing (*test_camerad*) | Static checks, CI gates |

Across all three layers, security is addressed through test_athenad.py and test_webrtcd.py, which test session handling, upload authorization, and failure behavior on network-facing surfaces.

## 3.3 Black-box, white-box, and object-oriented considerations

The strategy combines all three perspectives with examples drawn from the codebase:

- **Black-box**: *test_pandad_loopback.py* and *test_modeld.py* treat their respective daemons as opaque, meaning they inject inputs on one socket and verify outputs on another with no knowledge of internal implementation.
- **White-box**: *test_pandad_spi.py* sets an environment variable that activates fault injection code inside *spi.cc*'s *lltransfer()* function, exercising specific internal checksum and retry paths. The test is designed with full knowledge of those mechanisms.
- **Object-oriented**: *test_pandad_usbprotocol.cc* defines PandaTest as a subclass of Panda to access protected methods (*pack_can_buffer*, *unpack_can_buffer*) and internal buffers, testing the class through its own inheritance hierarchy. *test_athenad.py* tests the athenad dispatcher through its public interface using mock dependencies (*MockWebsocket*, *MockApi*).

### 3.4 Test prioritization and automation

Tests are prioritized by consequence of failure. *pandad* transport and safety tests are the highest priority because they gate physical car control. *modeld* output correctness is next because it feeds planning. *system/* logging, networking, and diagnostics are lower priority because failures degrade monitoring, not driving safety. Although *system/* has the broadest scope and the most test files of the three layers, individual failures there are less safety-critical, so it is prioritized by volume of coverage rather than urgency.

All unit and integration tests are automated via pytest and native test runners. System-level tests marked *@pytest.mark.tici* require comma 3 hardware and run in device CI or staging environments rather than desktop CI. This split prevents hardware-dependent tests from blocking the fast feedback loop while still ensuring they are executed before release.

## 4. Verification & Validation Approach

### 4.1 How verification and validation are conducted

This plan separates verification from validation to avoid mixing implementation checks with operational confidence claims.

Verification ensures that the implementation is built correctly, using unit/integration tests, static checks, and reproducible build evidence. Validation certifies that behavior remains acceptable in representative usage, using replay/staging scenarios and device-backed checks where needed to make sure everything is functioning as intended.

### 4.2 Alignment with software lifecycle

The V&V flow aligns with normal development phases:

- Development phase: targeted tests plus static/build checks on changed scope.
- Integration phase: cross-process and interface-level checks.
- Pre-release/staging phase: broader regression, soak/restart scenarios, and update-path validation.

This method of testing through the system's lifecycle prevents over-reliance on late-stage testing and supports faster feedback through each stage of development.

### 4.3 Traceability to expected behavior

Traceability is achieved through anchor mappings that link expected behavior to concrete evidence paths in the repository. Example anchors in our scope include:

- *Modeld* frame/message progression -> *selfdrive/modeld/tests/test_modeld.py*
- Pandad SPI/protocol boundaries -> *selfdrive/pandad/tests/test_pandad_spi.py*, *test_pandad_usbprotocol.cc*
- Manager lifecycle behavior -> *system/manager/test/test_manager.py*
- Logger record/encode/delete/upload behavior -> *system/loggerd/tests/** and native logger tests
- Camera behavior -> *system/camerad/test/**
- Athena/webrtc behavior -> *system/athena/tests/*, system/webrtc/tests/**
- Update behavior -> *system/updated/tests/**
- Sensor/location/hardware behavior -> *system/sensord/tests/*, system/ubloxd/tests/*, system/qcomgpsd/tests/*, system/hardware/tests/**

### 4.4 Standards alignment

This STP is structured to align with software-testing documentation standards, specifically IEEE 829 and ISO/IEC/IEEE 29119 concepts for test-plan structure, risk basis, lifecycle alignment, evidence traceability, and limitation disclosure [1], [2].

## 5. Test Environment

### 5.1 Tools, frameworks, and configuration

The environment combines local development, CI automation, and hardware-backed execution where needed to account for and test all parts of the system.

Some of the assumptions made in this plan to construct the core environment are:

- Python testing through *pytest*;
- Native build/testing via SCons and repository native targets;
- Static checks through repository tooling and CI;
- Execution contexts split across PC/dev, bench/device, and CI regression gates.

### 5.2 Test data considerations

Data selection is a core factor influencing testing confidence, comparable to tooling. For this scope, the expected data categories are:

- synthetic frame fixtures for deterministic *modeld* checks;

- Replay/recorded route data for regression stability;
- CAN and params fixtures for pandad and manager scenarios;
- Sensor/GPS traces for parser and fallback-path coverage.

# 6. Test Coverage & Metrics

## 6.1 Coverage criteria

Test coverage in this STP follows a risk-driven approach, meaning coverage is not applied uniformly across all subsystems, but given to high-impact areas, ensuring they are tested with verifiable evidence rather than focusing solely on achieving a single numerical coverage metric.

The coverage expectations throughout this STP are:

- Emphasizing code and branch coverage for high-impact modules;
- Ensuring interface coverage across messaging, lifecycle, and transport boundaries;
- Mapping each high-priority risk to at least one corresponding verification activity;
- Achieving full system and group-level coverage through automated tests or well-documented manual/staging validation.

## 6.2 Metrics used to evaluate test effectiveness

The following metrics are used to determine whether testing efforts are effectively improving system quality over time:

- Defect density by subsystem;
- Flaky test rate;
- Critical-path latency indicators (for example, p95 on camera/model flow);
- Logger failure indicators (drop/encode/upload);
- Restart recovery success rate;
- Escaped defects found after staging.

# 7. Regression & Nonfunctional Testing Strategy

## 7.1 Regression testing approach

Regression activity is triggered by code change scope, not by calendar alone. Changes under *modeld*, listed *pandad* files, or *system/* should execute their related scoped suites, plus build/static checks.

For higher-risk changes (model interfaces/assets, runtime orchestration, update logic), regression should be broadened with replay and cross-subsystem checks throughout to reduce false confidence from narrow test suites.

### 7.2 Planned nonfunctional tests

Nonfunctional testing covers quality characteristics that are often missed by pure correctness tests, leading to a more stable system overall. Some of these nonfunctional tests include:

- Performance checks for model, camera, and logger paths achieving proper object identification
- Stress/soak runs with manager-controlled process graphs ensuring function under load
- Reliability checks through process kill/restart and reconnect scenarios, ensuring the system is safe and reliable in all situations
- Security/privacy checks for athena/webrtc session and failure behavior
- Resource-behavior checks for disk pressure and uploader retries ensuring hardware stability
- Thermal/power checks on device-class hardware making sure the hardware is safe and reliable

## 8. Limitations & Risks

### 8.1 Known limitations of the test plan

Due to some of the limitations of possible tests this plan does not claim exhaustive real-world coverage. Important limitations of testing include incomplete replication of field variability, hardware dependence for parts of *system/*, and reduced confidence areas with lighter direct automation unless supplemented by staging evidence.

The update-path validation also remains operationally sensitive and thus requires controlled environments to provide accurate testing. The primary limitations of this plan include:

- Incomplete replication of field data: Test environments cannot fully reproduce the full range of real-world conditions, including changing road environments, network instability, sensor noise, thermal fluctuations, and unusual hardware states.
- Reduced confidence in lightly automated areas: Areas with limited direct automation may rely more heavily on manual checks, staging runs, or indirect evidence from adjacent tests. These areas therefore provide less repeatable confidence than heavily automated paths.
- Practical limits on combinational coverage: Because the system includes multiple interacting services, hardware configurations, and runtime conditions, it is not feasible to test every possible combination of states, failures, and dependencies.
- Dependence on representative test data: Confidence in model, sensor, logging, and replay-based checks depends on the quality and representativeness of the selected fixtures, traces, and recorded scenarios.

## 8.2 Potential challenges and mitigation strategies

There are numerous challenges that could be encountered, but since they are identified early, they can be effectively managed.
Some of these potential challenges include:

- Limited hardware availability: prioritize high-risk device checks first, to best utilize resources and ensure user safety
- Expensive full-stack runs:  use methods like layered tests and replay before broad runs to get a feel for how the full run will go
- Cross-language boundary defects: keep focused Cython/native boundary tests to have a clear division between languages
- Inter-process complexity: combine contract-style checks with manager integration to achieve better coverage
- Sensor/network nondeterminism: use traces, tolerances, and retry-aware checks once again increase coverage

# Reference

[1] IEEE Standards Association, *IEEE Std 829-2008: IEEE Standard for Software and System Test Documentation*. Available: https://standards.ieee.org/ieee/829/6803/

[2] ISO/IEC/IEEE, *ISO/IEC/IEEE 29119 Software Testing* (overview and parts). Available: https://www.iso.org/standard/45142.html

[3] comma.ai, *0.9.8 Release · commaai/openpilot*, GitHub. Available: https://github.com/commaai/openpilot/releases/tag/v0.9.8